# ELI — Lora Pipeline

ELI v2.5.13

October 2026

## Contents

**LoRA fine-tuning — audit & pipeline (2026-06-07)** **1**
What was found (audit) . . . 1
What was wired/fixed . . . 1
The deferred base-profile change, done . . . 2

## LoRA fine-tuning — audit & pipeline (2026-06-07)

### What was found (audit)

The LoRA mechanisms in `eli/learning/` were **individually correct but orphaned and phi-hardcoded**:

- **Algorithm — correct.** `lora_trainer.run_training` is a proper PEFT LoRA loop: tokenize (truncation/pad, `labels=input_ids`) → `AutoModelForCausalLM` (eager attn, fp16 on CUDA, gradient checkpointing) → `LoraConfig` → `get_peft_model` → `Trainer` with `DataCollatorForLanguageModeling(mlm=False)` → `trainer.train()` → `save_pretrained` (adapter + tokenizer). Safety contract is sound (dry-run default, `--execute` required, reviewed rows only, **GGUF never trained directly**, adapter never overwritten).
- **Stages exist** but as separate CLI scripts with no orchestrator: guard plan (`lora_trainer_guard`) → `training_preflight` → `dataset_builder` / `export_trainable_dataset` / `merge_reviewed_datasets` → `lora_trainer` → `lora_eval`. Nothing chained them.
- **Orphaned** — nothing outside `eli/learning/` called any of them (no executor action, router, GUI button, scheduled task, or self-upgrade hook). ELI could not trigger or even report on LoRA.
- **Not model-agnostic** — `target_modules` defaulted to phi-3's `["qkv_proj"]`.

### What was wired/fixed

- **Model-agnostic target modules** (`lora_trainer._resolve_target_modules`): derives LoRA targets from the LOADED architecture (scans for known projection Linear leaves: q/k/v/o_proj, qkv_proj, gate/up/down_proj, c_attn, query_key_value, ...), honours an explicit adapter-config override, and falls back to PEFT's architecture-agnostic `"all-linear"`. No hardcoded architecture on the train path.
- **Pipeline DAG** (`eli/learning/lora_pipeline.py::run_pipeline`): the explicit ordered chain **preflight → build_job → [train] → eval(inspect)**, reusing the existing modules. **Dry-run by default** (runs every gate, touches no GPU, writes no adapter — safe from chat); real training only `execute=True`.
- **Wired into ELI:**
  - `LORA_STATUS` action (read-only): preflight readiness per target.
  - `LORA_TRAIN` action: runs the pipeline DAG — **dry-run from chat**; real training only via the scheduled task / explicit GUI (execute flag not exposed in chat).

  - Router: “lora status” / “is lora ready” → LORA_STATUS; “train a lora” / “fine-tune yourself” / “run lora training” → LORA_TRAIN.
  - Scheduled `lora` kind (`_worker_lora`): “train a lora overnight [N steps]” → `SCHEDULE_TASK` runs `run_pipeline(execute=True)` unattended.
  - Manifest: 230 capabilities (includes LORA_STATUS + LORA_TRAIN).
- Tests: `tests/test_lora_pipeline.py` (target-module resolver, DAG order + dry-run no-train, both actions, routing, scheduled kind).

## The deferred base-profile change, done

The item recorded above as *“deferred to your go-ahead”* — making the base profile swappable to any HF causal-LM directory — is implemented, along with the GUI the pipeline docstring always claimed existed.

### 1. Targets are operator-declared (`eli/learning/target_registry.py`)

`ALLOWED_TARGETS = {"eli_phi", "eli_phi_ultra"}` plus `base_family must be phi3` was this machine’s two targets frozen into a redistributed product. The allowlist is still an allowlist — nothing trains unless explicitly declared — but declaring is now a runtime act written to `<data dir>/training/targets.json`. Family is **read** from the base model’s own `config.json` and checked against the declaration, so the check moved from *“is it Phi”* to *“is it what you said it was”*.

### 2. The review gate is reachable (`eli/learning/review_queue.py`)

The trainer always required human-approved, target-scoped rows. Nothing could produce them. Live state: **615 candidates, 0 trainable**, `will_train` permanently false. `ReviewQueue` adds assisted triage (auto-reject the provably unusable, flag what needs a human eye) and an approve/edit/reject loop. Triage **never approves** — that is a person’s act, pinned by test. Result on the live corpus: 615 → 20 auto-rejected, 309 clean, 286 flagged; approving the clean set produced `train_ready: true` for the first time.

### 3. Honest device selection

The flat `free_vram >= 10 GiB` floor refused a 1B on a 6 GB card and silently chose CPU, where a run takes days and looks like a hang. The floor is now the model’s own estimated need; the accelerator is named by vendor (NVIDIA / AMD-ROCm / Apple MPS / Intel XPU); and a refusal says what would fix it. QLoRA (4-bit nf4 + paged optimiser) lets a small card train the adapter at all.

### 4. Prompt format follows the base

`_format_example` hardcoded Phi-3’s `<|user|>/<|assistant|>`. Training a Qwen on Phi-3 turn markers teaches it tokens its chat template never uses — degrading the model rather than tuning it. The tokenizer’s own `chat_template` is now the source of truth, with the literal as fallback.

### 5. The GUI: Labs ▸ Training

Four steps — Hardware → Target → Data → Train — with live step/loss on a QThread, a dry-run readiness check, and cancel. See `gui.md`.

### 6. Paths

Runs, datasets and the registry follow `paths.learning_dir()` (source tree in dev, user data dir on a packaged install) instead of writing into a read-only mount.

**Still manual**

Merge → GGUF. `training/merge_and_convert.py` closes the loop but depends on `unsloth`, which is in no requirements file, so producing a usable model after a successful run remains a documented manual step. The Training tab says so plainly when a run finishes rather than implying the live model has changed.